

# Java OOP Study Notes: Classes, Objects, Fields & Methods

---

## 1. Class: Blueprint / Template

A **class** is a blueprint or template that defines the structure and behavior of objects. It describes what data an object will hold (fields) and what actions it can perform (methods).

### Syntax

```
[access_modifier] class ClassName {  
    // Fields (variables)  
    // Constructors  
    // Methods  
}
```

## Analogy: Class vs Object

### CLASS = BLUEPRINT

Like an architectural blueprint for a house:

- Defines structure (rooms, doors, windows)
- Defines features (plumbing, electrical)
- You can't live in a blueprint!

From ONE blueprint, you can build MANY houses

|



### OBJECTS = ACTUAL HOUSES

| House 1      | House 2      | House 3       |
|--------------|--------------|---------------|
| (Red color)  | (Blue color) | (White color) |
| (3 bedrooms) | (2 bedrooms) | (4 bedrooms)  |

Each house is built from the same blueprint  
but has its own unique characteristics

## Basic Class Example

```
// Class definition - the blueprint
class Student {
    // Fields (what a student HAS)
    String name;
    int age;
    String studentId;

    // Constructor (how to create a student)
    Student(String name, int age, String studentId) {
        this.name = name;
        this.age = age;
        this.studentId = studentId;
    }

    // Methods (what a student CAN DO)
    void study() {
        System.out.println(name + " is studying");
    }

    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("ID: " + studentId);
    }
}
```

# Class Components

| Component      | Purpose                | Example                                 |
|----------------|------------------------|-----------------------------------------|
| Fields         | Store data/state       | <code>String name;</code>               |
| Constructors   | Initialize objects     | <code>Student(String name) {...}</code> |
| Methods        | Define behavior        | <code>void study() {...}</code>         |
| Nested Classes | Classes inside classes | <code>class Inner {...}</code>          |
| Blocks         | Initialization code    | <code>static {...} or {...}</code>      |

## 2. Object: Instance of Class

An **object** is a concrete instance of a class. It exists in memory and has actual values for the fields defined in the class.

### Creating Objects: The `new` Keyword

```
ClassName objectName = new ClassName(arguments);
```

### What Happens When You Create an Object?

```
Student s1 = new Student("Aslam", 20, "STU001");  
|           |           |  
|           |           |  
|           |           |  
|           |           |  
└─ Reference variable (stores address)  
|  
└─ new keyword allocates memory  
|  
|           |           |  
|           |           |  
└─ Constructor arguments
```

Step 1: `new Student(...)`

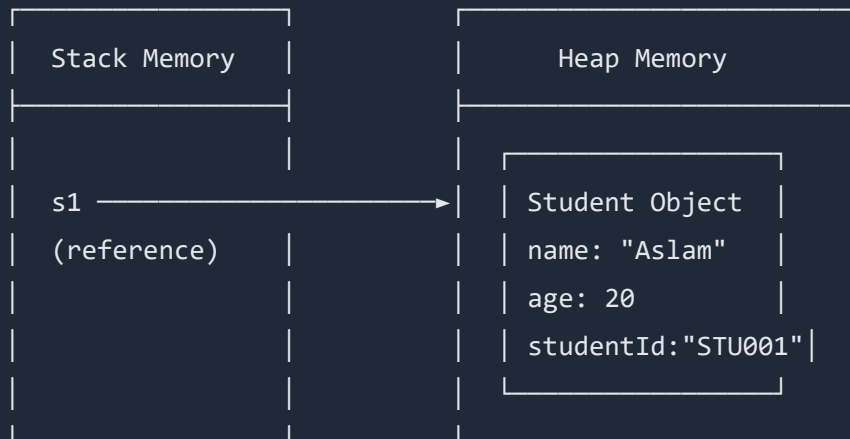
- Allocates memory in HEAP
- Creates object with fields

Step 2: `Student(...)` constructor runs

- Initializes fields with values

Step 3: Reference assigned

- `s1` stores the memory address



## Creating Multiple Objects

```
public class Main {  
    public static void main(String[] args) {  
        // Creating multiple objects from same class  
        Student s1 = new Student("Aslam", 20, "STU001");  
        Student s2 = new Student("Rahim", 22, "STU002");  
        Student s3 = new Student("Karim", 19, "STU003");  
  
        // Each object has its own data  
        s1.displayInfo();  
        System.out.println();  
        s2.displayInfo();  
        System.out.println();  
        s3.displayInfo();  
  
        // Each object can perform actions independently  
        s1.study(); // Aslam is studying  
        s2.study(); // Rahim is studying  
    }  
}
```

### Output:

```
Name: Aslam  
Age: 20  
ID: STU001  
  
Name: Rahim  
Age: 22  
ID: STU002  
  
Name: Karim  
Age: 19  
ID: STU003  
  
Aslam is studying  
Rahim is studying
```

## Object vs Reference

```
Student s1 = new Student("Aslam", 20, "STU001");
Student s2 = s1; // s2 points to SAME object as s1

s2.name = "Changed";

System.out.println(s1.name); // Output: "Changed"
// Both s1 and s2 reference the SAME object!
```



Both references point to the SAME object

## Null Reference

```
Student s1 = null; // s1 doesn't point to any object

// s1.study(); // NullPointerException!

// Always check for null before using
if (s1 != null) {
    s1.study();
}
```

---

## 3. Fields / Variables

Java has three types of variables based on where they are declared:

| Type              | Declared In                       | Scope        | Default Value | Memory      |
|-------------------|-----------------------------------|--------------|---------------|-------------|
| Instance Variable | Class (outside methods)           | Object       | Yes           | Heap        |
| Static Variable   | Class (with <code>static</code> ) | Class        | Yes           | Method Area |
| Local Variable    | Method/Block                      | Method/Block | No            | Stack       |

## 3.1 Instance Variables

**Instance variables** belong to an object. Each object has its own copy.

```
class Student {  
    // Instance variables - each object gets its own copy  
    String name;      // Default: null  
    int age;          // Default: 0  
    double gpa;       // Default: 0.0  
    boolean active;   // Default: false  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Student s2 = new Student();  
  
        s1.name = "Aslam";  
        s1.age = 20;  
  
        s2.name = "Rahim";  
        s2.age = 22;  
  
        // Each object has SEPARATE copies  
        System.out.println(s1.name); // Aslam  
        System.out.println(s2.name); // Rahim  
    }  
}
```



## Characteristics of Instance Variables

| Feature       | Description                |
|---------------|----------------------------|
| Scope         | Throughout the class       |
| Lifetime      | As long as object exists   |
| Default Value | Yes (0, null, false, etc.) |
| Access        | Through object reference   |
| Memory        | Heap (inside object)       |
| One per       | Object                     |

## Default Values for Instance Variables

```
class Defaults {  
    byte b;        // 0  
    short s;       // 0  
    int i;         // 0  
    long l;        // 0L  
    float f;       // 0.0f  
    double d;      // 0.0  
    char c;        // '\u0000' (null character)  
    boolean bool;  // false  
    String str;    // null  
    Object obj;    // null  
}
```

## 3.2 Static Variables (Class Variables)

**Static variables** belong to the class, not individual objects. All objects share ONE copy.

```
class Student {  
    // Instance variable - each object has its own  
    String name;  
  
    // Static variable - shared by ALL objects  
    static String schoolName = "ABC School";  
    static int totalStudents = 0;  
  
    Student(String name) {  
        this.name = name;  
        totalStudents++; // Increment shared counter  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // Access static variable WITHOUT object  
        System.out.println("School: " + Student.schoolName);  
  
        Student s1 = new Student("Aslam");  
        Student s2 = new Student("Rahim");  
        Student s3 = new Student("Karim");  
  
        // All objects share the same totalStudents  
        System.out.println("Total: " + Student.totalStudents); // 3  
  
        // Can also access through object (not recommended)  
        System.out.println("Total: " + s1.totalStudents); // 3  
    }  
}
```

# Memory Diagram: Static vs Instance



## Characteristics of Static Variables

| Feature       | Description                              |
|---------------|------------------------------------------|
| Scope         | Throughout the class                     |
| Lifetime      | As long as class is loaded               |
| Default Value | Yes (same as instance)                   |
| Access        | Through class name (preferred) or object |
| Memory        | Method Area (class memory)               |
| One per       | Class (shared by all objects)            |

## When to Use Static Variables

```
class BankAccount {  
    // Instance - unique per account  
    private String accountNumber;  
    private double balance;  
  
    // Static - shared across all accounts  
    private static double interestRate = 0.05; // Same for all  
    private static int totalAccounts = 0;      // Counter  
    private static String bankName = "National Bank"; // Same for all  
  
    BankAccount(String accountNumber) {  
        this.accountNumber = accountNumber;  
        this.balance = 0;  
        totalAccounts++;  
    }  
  
    // Static method to change rate for ALL accounts  
    static void setInterestRate(double rate) {  
        interestRate = rate;  
    }  
}
```

---

### 3.3 Local Variables

**Local variables** are declared inside methods, constructors, or blocks. They exist only during execution of that block.

```

class Calculator {

    void calculate() {
        // Local variables - exist only in this method
        int a = 10;
        int b = 20;
        int sum = a + b;  // Must be initialized before use!

        System.out.println("Sum: " + sum);
    }  // a, b, sum are destroyed here

    void anotherMethod() {
        // Cannot access a, b, sum here - they don't exist!
        // System.out.println(sum);  // ERROR!

        int sum = 100;  // This is a different 'sum'
    }
}

```

## Characteristics of Local Variables

| Feature       | Description                               |
|---------------|-------------------------------------------|
| Scope         | Only within the block where declared      |
| Lifetime      | Until block execution completes           |
| Default Value | <b>NO!</b> Must be initialized before use |
| Access        | Only within the block                     |
| Memory        | Stack                                     |
| One per       | Method invocation                         |

## Local Variables MUST Be Initialized

```
void example() {  
    int x;           // Declared but not initialized  
    // System.out.println(x); // ERROR! Variable 'x' might not have been initialized  
  
    x = 10;          // Now initialized  
    System.out.println(x); // OK  
}
```

## Local Variables in Different Blocks

```
void demo() {  
    int a = 10; // Method-level local variable  
  
    if (true) {  
        int b = 20; // Block-level local variable  
        System.out.println(a); // OK - a is accessible  
        System.out.println(b); // OK - b is accessible  
    }  
  
    System.out.println(a); // OK - a still exists  
    // System.out.println(b); // ERROR! b is out of scope  
  
    for (int i = 0; i < 5; i++) {  
        int c = i * 2; // Loop-level local variable  
        System.out.println(c);  
    }  
  
    // System.out.println(i); // ERROR! i is out of scope  
    // System.out.println(c); // ERROR! c is out of scope  
}
```

### 3.4 Variable Comparison Table

| Feature       | Instance Variable       | Static Variable         | Local Variable      |
|---------------|-------------------------|-------------------------|---------------------|
| Keyword       | None                    | <code>static</code>     | None                |
| Location      | Class (outside methods) | Class (outside methods) | Inside method/block |
| Memory        | Heap (in object)        | Method Area             | Stack               |
| Default Value | Yes                     | Yes                     | <b>No</b>           |
| Scope         | Entire class            | Entire class            | Block only          |
| Lifetime      | Object lifetime         | Class lifetime          | Block execution     |
| Accessed via  | Object reference        | Class name              | Directly            |
| Copies        | One per object          | One per class           | One per invocation  |

## Complete Example

```
class Employee {
    // Static variable - shared by all
    static String companyName = "Tech Corp";
    static int employeeCount = 0;

    // Instance variables - unique per object
    String name;
    int id;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
        this.id = ++employeeCount;
    }

    void calculateBonus() {
        // Local variables - exist only in this method
        double bonusRate = 0.10;
        double bonus = salary * bonusRate;
        double totalPay = salary + bonus;

        System.out.println(name + "'s bonus: $" + bonus);
        System.out.println("Total pay: $" + totalPay);
    }

    void displayInfo() {
        System.out.println("Company: " + companyName); // Static
        System.out.println("ID: " + id);               // Instance
        System.out.println("Name: " + name);           // Instance
        System.out.println("Salary: $" + salary);      // Instance
    }

    static void showCompanyInfo() {
        System.out.println("Company: " + companyName);
        System.out.println("Total Employees: " + employeeCount);
        // System.out.println(name); // ERROR! Can't access instance variable
    }
}
```



```
}

public class Main {
    public static void main(String[] args) {
        // Access static without object
        System.out.println("Company: " + Employee.companyName);

        Employee e1 = new Employee("Aslam", 50000);
        Employee e2 = new Employee("Rahim", 60000);

        e1.displayInfo();
        e1.calculateBonus();

        System.out.println();

        e2.displayInfo();
        e2.calculateBonus();

        System.out.println();

        // Static method call
        Employee.showCompanyInfo();
    }
}
```

---

## 4. Methods

Methods define the behavior of a class — what actions objects can perform.

## Method Syntax

```
[access_modifier] [static] [final] returnType methodName(parameters) {  
    // Method body  
    return value; // If not void  
}
```

---

### 4.1 Instance Methods

**Instance methods** belong to objects. They can access both instance and static members.

```
class Calculator {
    // Instance variable
    private int lastResult;

    // Instance methods - called on objects
    int add(int a, int b) {
        lastResult = a + b;
        return lastResult;
    }

    int subtract(int a, int b) {
        lastResult = a - b;
        return lastResult;
    }

    int getLastResult() {
        return lastResult;
    }

    // Instance method can access static members
    void showInfo() {
        System.out.println("Calculator v1.0");
        System.out.println("Last result: " + lastResult);
    }
}

public class Main {
    public static void main(String[] args) {
        // Must create object to call instance methods
        Calculator calc = new Calculator();

        int sum = calc.add(10, 5);
        System.out.println("Sum: " + sum); // 15

        int diff = calc.subtract(10, 5);
        System.out.println("Diff: " + diff); // 5

        calc.showInfo();
    }
}
```

```
}  
}
```

## Characteristics of Instance Methods

| Feature    | Description                                   |
|------------|-----------------------------------------------|
| Call       | Through object: <code>obj.method()</code>     |
| Can access | Instance variables, static variables          |
| Can call   | Instance methods, static methods              |
| Can use    | <code>this</code> keyword                     |
| Memory     | Method code in Method Area, executes on Stack |

## 4.2 Static Methods

**Static methods** belong to the class, not objects. They can only directly access static members.

```
class MathUtils {
    // Static variable
    private static final double PI = 3.14159;

    // Static methods - called on class, not object
    static int add(int a, int b) {
        return a + b;
    }

    static int max(int a, int b) {
        return (a > b) ? a : b;
    }

    static double circleArea(double radius) {
        return PI * radius * radius;
    }

    static boolean isEven(int num) {
        return num % 2 == 0;
    }

    static int factorial(int n) {
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }
}

public class Main {
    public static void main(String[] args) {
        // Call static methods WITHOUT creating object
        int sum = MathUtils.add(10, 5);
        int maximum = MathUtils.max(10, 5);
        double area = MathUtils.circleArea(5);
        boolean even = MathUtils.isEven(10);
        int fact = MathUtils.factorial(5);

        System.out.println("Sum: " + sum);           // 15
        System.out.println("Max: " + maximum);       // 10
        System.out.println("Area: " + area);         // 78.53975
        System.out.println("Is Even: " + even);      // true
    }
}
```

```
        System.out.println("5! = " + fact);        // 120
    }
}
```

## Characteristics of Static Methods

| Feature    | Description                                    |
|------------|------------------------------------------------|
| Call       | Through class: <code>ClassName.method()</code> |
| Can access | Only static variables (directly)               |
| Can call   | Only static methods (directly)                 |
| Cannot use | <code>this</code> keyword (no object context)  |
| Use case   | Utility methods, factory methods               |

## Static Method Restrictions

```
class Demo {
    int instanceVar = 10;
    static int staticVar = 20;

    void instanceMethod() {
        System.out.println("Instance method");
    }

    static void staticMethod() {
        // Can access static members
        System.out.println(staticVar); // OK

        // CANNOT directly access instance members
        // System.out.println(instanceVar); // ERROR!
        // instanceMethod(); // ERROR!
        // System.out.println(this); // ERROR!

        // CAN access instance members through an object
        Demo obj = new Demo();
        System.out.println(obj.instanceVar); // OK
        obj.instanceMethod(); // OK
    }
}
```

---

## 4.3 Instance vs Static Methods Comparison

| Feature               | Instance Method              | Static Method                   |
|-----------------------|------------------------------|---------------------------------|
| Keyword               | None                         | <code>static</code>             |
| Belongs to            | Object                       | Class                           |
| Call syntax           | <code>object.method()</code> | <code>ClassName.method()</code> |
| Access instance vars  | Yes                          | No (only through object)        |
| Access static vars    | Yes                          | Yes                             |
| Use <code>this</code> | Yes                          | No                              |
| Common use            | Object behavior              | Utility functions               |



## When to Use Each

```
class BankAccount {  
    // Instance variables  
    private String accountNumber;  
    private double balance;  
  
    // Static variables  
    private static double interestRate = 0.05;  
    private static int totalAccounts = 0;  
  
    // Constructor  
    BankAccount(String accountNumber) {  
        this.accountNumber = accountNumber;  
        this.balance = 0;  
        totalAccounts++;  
    }  
  
    // =====  
    // INSTANCE METHODS - work with object's data  
    // =====  
  
    void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount; // Access instance variable  
        }  
    }  
  
    void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
        }  
    }  
  
    double getBalance() {  
        return balance;  
    }  
  
    void addInterest() {  
        balance += balance * interestRate; // Uses both instance and static
```

```

    }

    void displayInfo() {
        System.out.println("Account: " + accountNumber);
        System.out.println("Balance: $" + balance);
    }

    // =====
    // STATIC METHODS - work with class-level data
    // =====

    static void setInterestRate(double rate) {
        interestRate = rate; // Only static access
    }

    static double getInterestRate() {
        return interestRate;
    }

    static int getTotalAccounts() {
        return totalAccounts;
    }

    static void printBankInfo() {
        System.out.println("Bank Interest Rate: " + (interestRate * 100) + "%");
        System.out.println("Total Accounts: " + totalAccounts);
    }
}

public class Main {
    public static void main(String[] args) {
        // Static methods - no object needed
        BankAccount.setInterestRate(0.06);
        BankAccount.printBankInfo();

        System.out.println();

        // Instance methods - need object
        BankAccount acc1 = new BankAccount("ACC001");
        BankAccount acc2 = new BankAccount("ACC002");
    }
}

```

```
        acc1.deposit(1000);
        acc1.addInterest();
        acc1.displayInfo();

        System.out.println();

        acc2.deposit(2000);
        acc2.withdraw(500);
        acc2.displayInfo();

        System.out.println();

        // Static method shows updated count
        System.out.println("Total accounts: " + BankAccount.getTotalAccounts());
    }
}
```

## Output:

```
Bank Interest Rate: 6.0%
Total Accounts: 0

Account: ACC001
Balance: $1060.0

Account: ACC002
Balance: $1500.0

Total accounts: 2
```

## 5. Complete Example: All Concepts Together

```
class Product {  
    // =====  
    // STATIC VARIABLES - Shared by all products  
    // =====  
    private static String storeName = "Tech Store";  
    private static int totalProducts = 0;  
    private static double taxRate = 0.10;  
  
    // =====  
    // INSTANCE VARIABLES - Unique per product  
    // =====  
    private int productId;  
    private String name;  
    private double price;  
    private int quantity;  
  
    // =====  
    // CONSTRUCTOR  
    // =====  
    Product(String name, double price, int quantity) {  
        this.productId = ++totalProducts; // Auto-increment ID  
        this.name = name;  
        this.price = price;  
        this.quantity = quantity;  
    }  
  
    // =====  
    // STATIC METHODS - Class-level operations  
    // =====  
  
    static void setTaxRate(double rate) {  
        taxRate = rate;  
    }  
  
    static double getTaxRate() {  
        return taxRate;  
    }  
}
```

```

static int getTotalProducts() {
    return totalProducts;
}

static void printStoreInfo() {
    System.out.println("=====");
    System.out.println("||  Store: " + storeName);
    System.out.println("||  Tax Rate: " + (taxRate * 100) + "%");
    System.out.println("||  Total Products: " + totalProducts);
    System.out.println("=====");
}

// =====
// INSTANCE METHODS - Object-level operations
// =====

double calculateTotal() {
    // LOCAL VARIABLES - exist only in this method
    double subtotal = price * quantity;
    double tax = subtotal * taxRate; // Uses static variable
    double total = subtotal + tax;

    return total;
}

void applyDiscount(double percentage) {
    // LOCAL VARIABLE
    double discount = price * (percentage / 100);
    price -= discount;
    System.out.println("Applied " + percentage + "% discount. New price: $" + price);
}

void addStock(int amount) {
    if (amount > 0) {
        quantity += amount;
        System.out.println("Added " + amount + " units. New quantity: " + quantity);
    }
}

void removeStock(int amount) {

```

```

        if (amount > 0 && amount <= quantity) {
            quantity -= amount;
            System.out.println("Removed " + amount + " units. New quantity: " + quantity);
        } else {
            System.out.println("Cannot remove " + amount + " units. Only " + quantity + " available");
        }
    }

    void displayInfo() {
        System.out.println("┌───────────────────────────────────┐");
        System.out.println("│ Product ID: " + productId);
        System.out.println("│ Name: " + name);
        System.out.println("│ Price: $" + price);
        System.out.println("│ Quantity: " + quantity);
        System.out.println("│ Subtotal: $" + (price * quantity));
        System.out.println("│ Tax: $" + (price * quantity * taxRate));
        System.out.println("│ Total: $" + calculateTotal());
        System.out.println("└───────────────────────────────────┘");
    }

    // Getters
    int getProductId() { return productId; }
    String getName() { return name; }
    double getPrice() { return price; }
    int getQuantity() { return quantity; }
}

// =====
// MAIN CLASS
// =====

public class Main {
    public static void main(String[] args) {
        // =====
        // USING STATIC METHODS (no object needed)
        // =====
        Product.setTaxRate(0.08); // Set 8% tax
        Product.printStoreInfo();

        System.out.println();
    }
}

```

```

// =====
// CREATING OBJECTS (instances of class)
// =====
Product laptop = new Product("Gaming Laptop", 1200.00, 2);
Product phone = new Product("Smartphone", 800.00, 5);
Product tablet = new Product("Tablet", 500.00, 3);

// =====
// USING INSTANCE METHODS (need object)
// =====
System.out.println("=== Initial Products ===\n");
laptop.displayInfo();
phone.displayInfo();
tablet.displayInfo();

System.out.println("\n=== Modifying Products ===\n");

laptop.applyDiscount(10); // 10% off
laptop.displayInfo();

phone.addStock(10);
phone.removeStock(3);
phone.displayInfo();

// =====
// STATIC METHOD SHOWS UPDATED COUNT
// =====
System.out.println("\n=== Store Summary ===\n");
Product.printStoreInfo();

// =====
// POLYMORPHISM - Array of objects
// =====
System.out.println("\n=== All Products ===\n");
Product[] inventory = { laptop, phone, tablet };

double grandTotal = 0;
for (Product p : inventory) {
    System.out.println(p.getName() + ": $" + p.calculateTotal());
    grandTotal += p.calculateTotal();
}

```

```
    }  
    System.out.println("_____");  
    System.out.println("Grand Total: $" + grandTotal);  
}  
}
```

## Output:

```
┌───────────────────────────────────┐  
│ Store: Tech Store                  │  
│ Tax Rate: 8.0%                    │  
│ Total Products: 0                  │  
└───────────────────────────────────┘
```

=== Initial Products ===

```
┌───────────────────────────────────┐  
│ Product ID: 1                     │  
│ Name: Gaming Laptop               │  
│ Price: $1200.0                    │  
│ Quantity: 2                       │  
│ Subtotal: $2400.0                 │  
│ Tax: $192.0                       │  
│ Total: $2592.0                    │  
└───────────────────────────────────┘
```

... (more output)

=== Store Summary ===

```
┌───────────────────────────────────┐  
│ Store: Tech Store                  │  
│ Tax Rate: 8.0%                    │  
│ Total Products: 3                  │  
└───────────────────────────────────┘
```



## 6. Key Points to Remember

### Class & Object

1. **Class** = Blueprint/template that defines structure and behavior
2. **Object** = Instance of class with actual data in memory
3. Use `new` keyword to create objects
4. Multiple objects from same class have separate instance data

### Variables

5. **Instance variables** — belong to object, one per object
6. **Static variables** — belong to class, shared by all objects
7. **Local variables** — exist only in method/block, NO default value
8. Instance and static variables have default values

### Methods

9. **Instance methods** — called on object, can access all members
10. **Static methods** — called on class, can only directly access static members
11. Static methods cannot use `this` keyword
12. Use static for utility functions that don't need object state

### Memory

13. Static variables → Method Area
14. Instance variables → Heap (inside object)
15. Local variables → Stack
16. Reference variables store addresses, not objects

```
'; triggerDownload(docTitle.replace(/^[^a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html; charset=utf-8'); }  
function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); //  
Force light mode for PDF (dark backgrounds print poorly) const wasDark =
```

```

document.documentElement.classList.contains('dark'); if (wasDark)
document.documentElement.classList.remove('dark'); const element =
document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename:
docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale:
1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' },
pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if
(wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid
rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme:
dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre
code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !==
'mermaid') { hljs.highlightElement(block); } }); updateHljsTheme(isDark); // Render mermaid diagrams try {
mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const
mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i <
mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const
code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const
container = document.createElement('div'); container.className = 'mermaid-container';
container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render
failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function
updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark;
document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */
window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const
isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark);
updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme:
isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) {} });

```